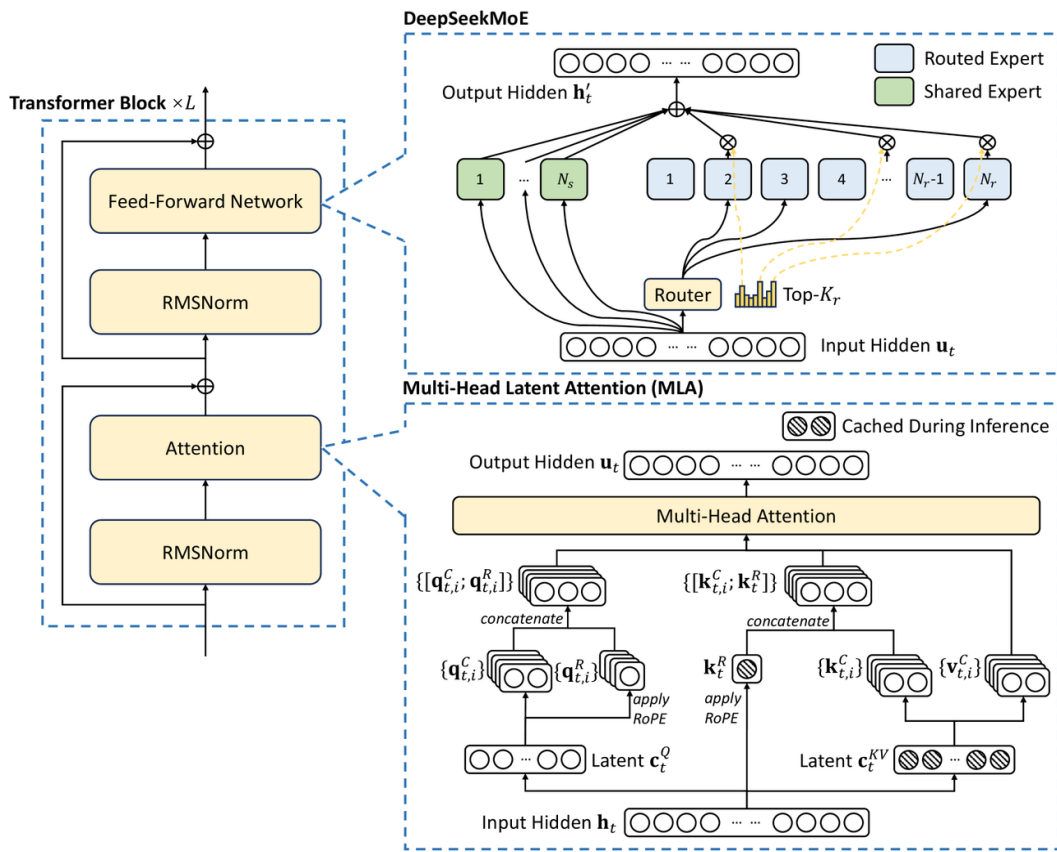




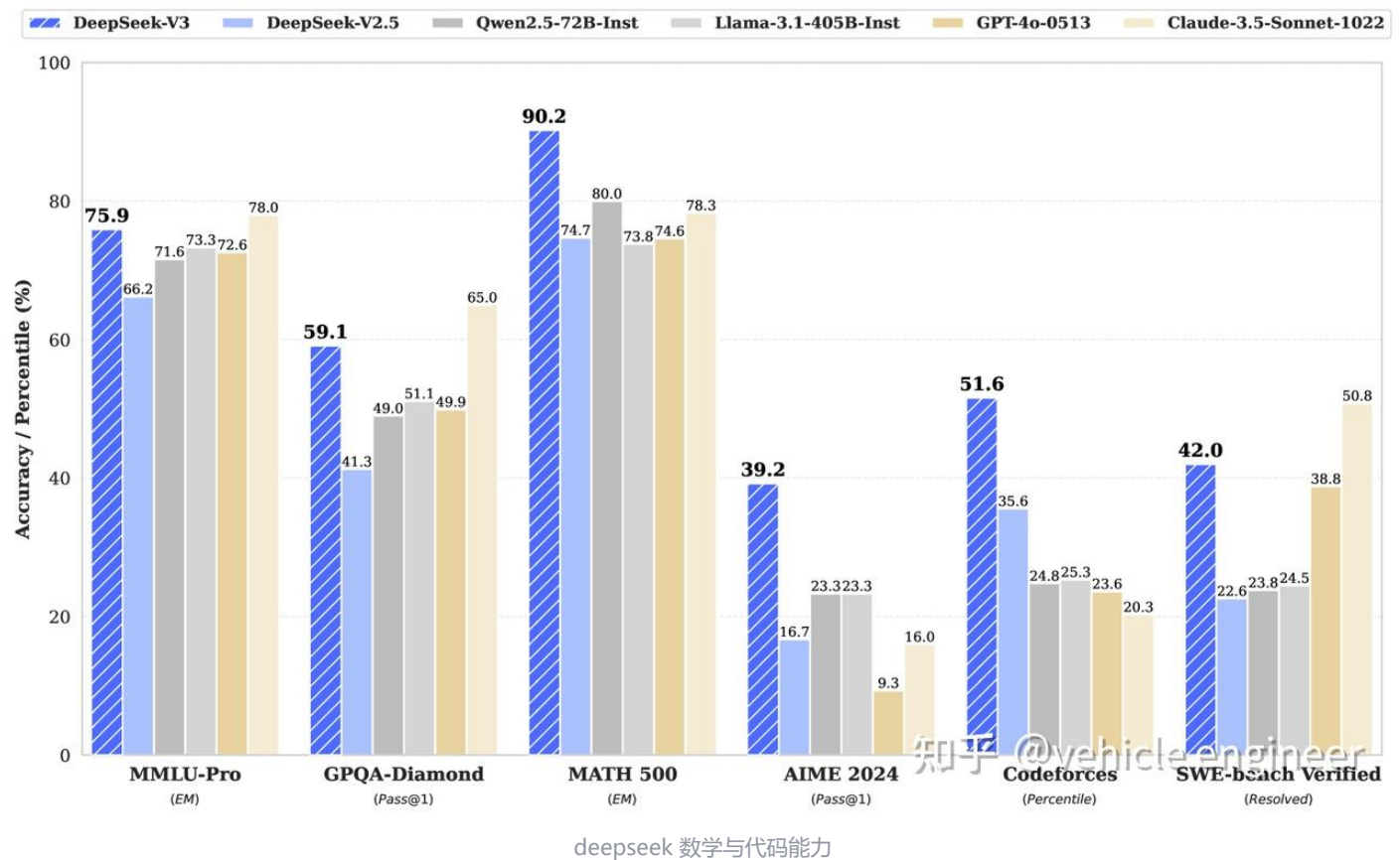
## Deepseek v3 技术报告万字硬核解读



学车辆算法工程师  
学车辆的算法工程师

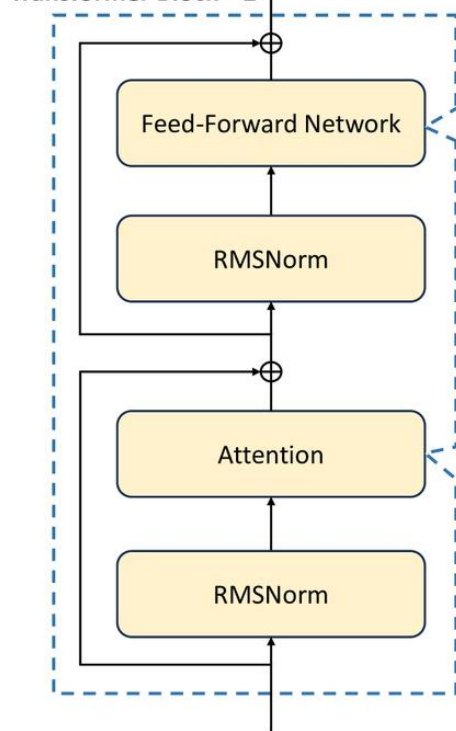
226 人赞同了该文章

### 1. 模型基本性能

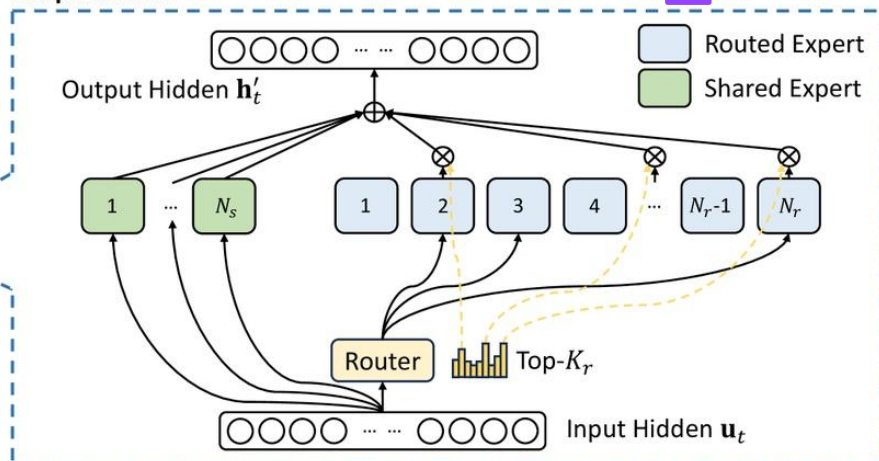


### 2. 模型架构层 (无本质创新, 这里引用讲解为主)

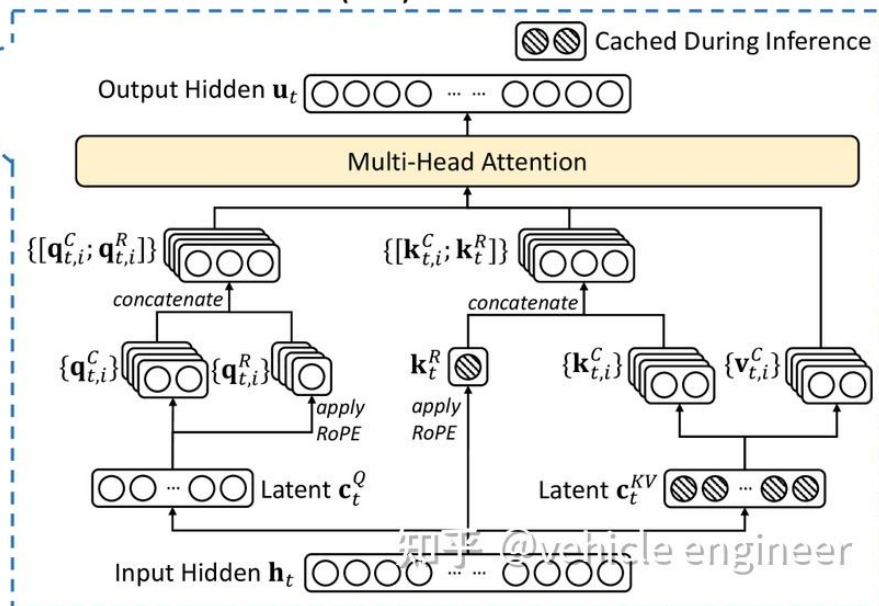
Transformer Block  $\times L$



DeepSeekMoE



Multi-Head Latent Attention (MLA)



deepseekv3 架构图

## 2.1 Config

```
"aux_loss_alpha": 0.001,
"ep_size": 1,
"hidden_act": "silu", # 激活函数
"hidden_size": 7168, # hidden size
"initializer_range": 0.02,
"intermediate_size": 18432, # 这个是mlp中间层大小
"kv_lora_rank": 512, # mla的kv低秩维度
"max_position_embeddings": 163840, # 最大长度
"moe_intermediate_size": 2048, # 专家的中间维度
"n_group": 8,
"n_routed_experts": 256, # 非共享专家数
"n_shared_experts": 1, # 共享专家数
"num_attention_heads": 128, # 注意力头数
"num_experts_per_tok": 8, # 每个token激活抓头数
"num_hidden_layers": 61, # transformer层数
"num_key_value_heads": 128, # k, v头数
"pretraining_tp": 1, # 预训练tp数
"q_lora_rank": 1536, # query的lora维度
"qk_nope_head_dim": 128, # qk的无rope维度
"qk_rope_head_dim": 64, # qk的rope维度
"quantization_config": {
  "activation_scheme": "dynamic", # activation动态量化
  "fmt": "e4m3", # 指数4位尾数3位
  "quant_method": "fp8", # 量化数据结构
}
```

```

"weight_block_size": [
    128, # 量化分组数，即一个128*128的矩阵共享一组scaling和bias
    128
],
},
"rms_norm_eps": 1e-06, #rms 的参数
"rope_scaling": {# yarn的上下文长度拓展参数
    "beta_fast": 32,
    "beta_slow": 1,
    "factor": 40,
    "scale": 1.0,
    "mscale_all_dim": 1.0,
    "original_max_position_embeddings": 4096,
    "type": "yarn"
},
"rope_theta": 10000, # theta的大小
"router_scaling_factor": 2.5, #router的放大系数
"scoring_fun": "sigmoid",
"seq_aux": true,
"tie_word_embeddings": false, #lm_head和embedding不共享
"topk_group": 4, #激活专家group
"v_head_dim": 128, #v的维度
"vocab_size": 129280 #词表数

```

## 2.2 分词



采用 **BBPE 技术**

[zhuanlan.zhihu.com](https://zhuanlan.zhihu.com)

## 2.3 MLA

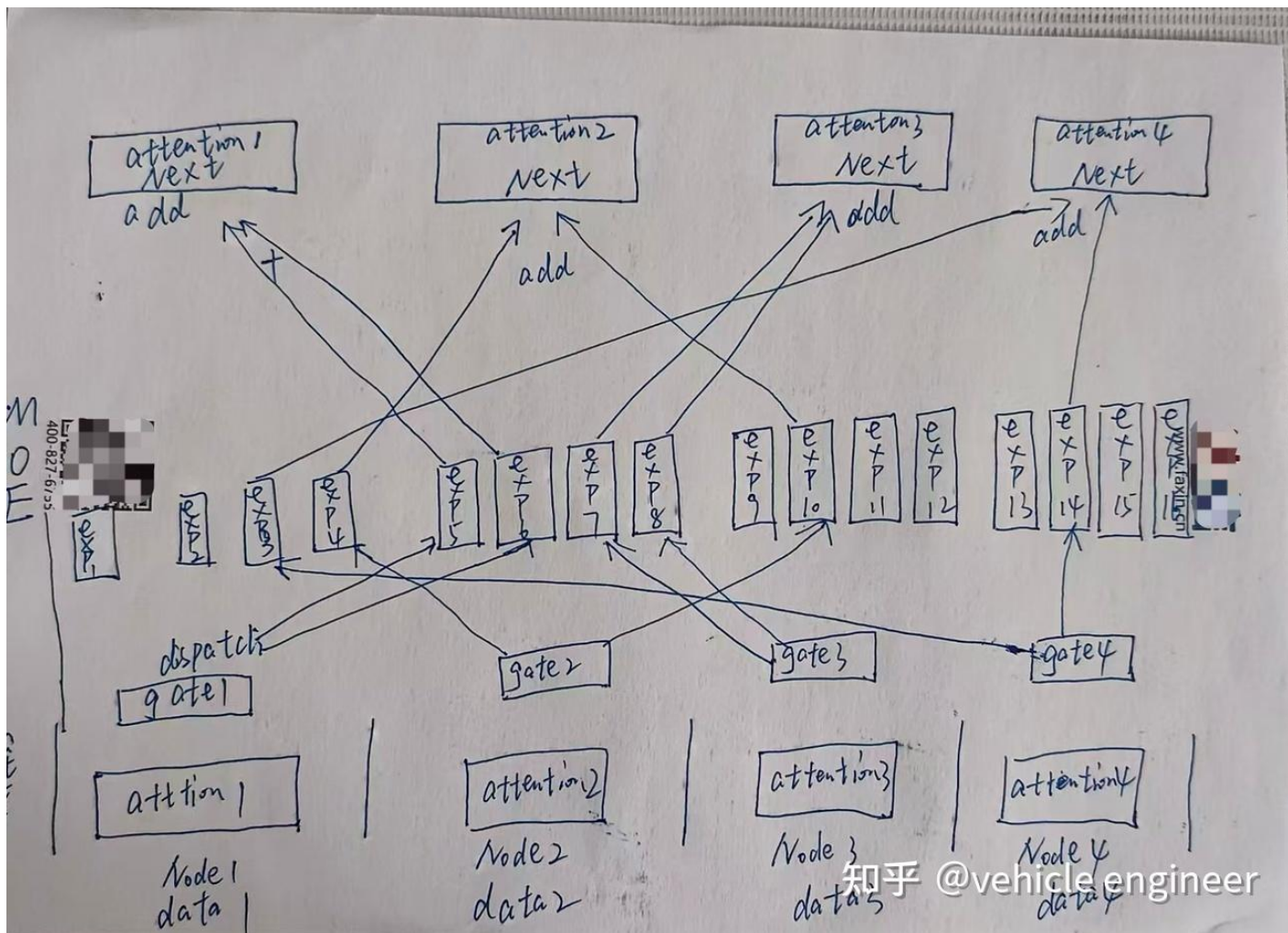
关于MLA我在另外一篇文章中有详细的解读

[【薪火相传】MiniCPM的三代注意力演进 - 知乎](#)

## 2.4 Share MOE

1. 专家头包括 **Share 专家** 和 **Router 专家**。
2. **Share 专家** 是**一直激活的**，即输入的 token 都会被 Share 专家头计算。
3. **Router 专家头** 会先和上图中的  $u_t$  计算亲和度（代码中直接用一个 Linear 层进行投影），选择 top-k 各专家进行推理。（代码中推理计算 top-k 时，会先将 N 各专家进行分组为 `n_groups`，将每个组中 top-2 各专家的亲和力加起来，算出亲和力最高的 `top_k_group` 个组，然后在这些组里选 top-k 个专家）。
4. 最终将所有的 Share 输出和 Router 专家进行**亲和度加权相加**，得到 MoE 层的输出。





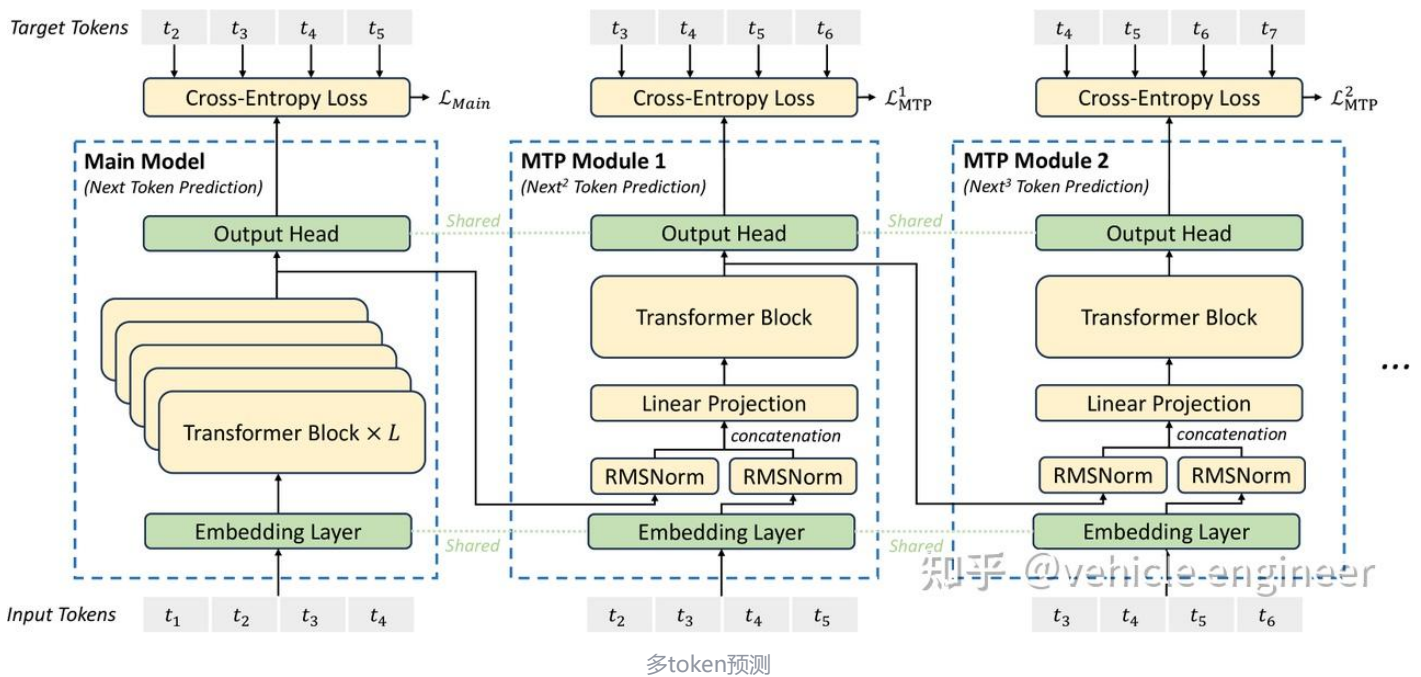
MOE前向DP与PP前向





### 3. 训练方法创新

#### 3.1 MTP



#### MTP 代码复现

```
## MTP 代码
## MLA和MOE代码省略, 其他基本完善
import torch
import torch.nn as nn

class RMSNorm(nn.Module):
    def __init__(self, dim: int, eps: float = 1e-8):
        super().__init__()
        self.eps = eps
        self.weight = nn.Parameter(torch.ones(dim)) # 可学习的缩放参数
    def _norm(self, x: torch.Tensor):
        return x * torch.rsqrt(x.pow(2).mean(-1, keepdim=True) + self.eps)
    def forward(self, x: torch.Tensor):
        return self.weight * self._norm(x.float()).type_as(x)

class share_embedding(nn.Module, config):
    def __init__(self, config: DeepseekV3Config):
        super().__init__()
        self.embed_tokens = nn.Embedding(
            config.vocab_size, config.hidden_size
        )
    def forward(self, input_ids):
        return self.embed_tokens(input_ids)

class share_output_head(nn.Module, config):
    def __init__(self, config: DeepseekV3Config):
        super().__init__()
        self.lm_head = nn.Linear(config.hidden_size, config.vocab_size)
    def forward(self, transformer_hidden):
        return self.lm_head(transformer_hidden)
```

```

class transformer_block(nn.Module, config):
    def __init__(self, config: DeepseekV3Config):
        super().__init__()
        self.mla = MLA(config)
        self.moe = MOE(config)
        self.pre_RMSNorm = RMSNorm(config)
        self.post_RMSNorm = RMSNorm(config)
    def forward(self, input_hidden):
        out_logits = self.pre_RMSNorm(input_hidden)
        out_logits = self.mla(out_logits)
        out_logits = self.post_RMSNorm(out_logits)
        moe_input = input_hidden+out_logits
        moe_output = self.moe(moe_input)
        block_output = moe_output+moe_input
        return block_output

class MTP(nn.Module, config):
    def __init__(self, config:Deepseekv3Config):
        super().__init__()
        self.RMSNorm_right = RMSNorm(config)
        self.RMSNorm_left = RMSNorm(config)
        self.transformer = transformer_block(config)
        self.proj = nn.Linear(2*config.hidden_size, config.hidden_size)
    def forward(self, last_block_out, input_ids_truncated, share_embedding, share_lm_head):
        last_norm_out = self.RMSNorm_right(last_block_out)
        embedding_trunc = share_embedding(input_ids_truncated)
        concat_input = torch.cat((last_norm_out, embedding_trunc), dim=-1)
        proj_out = self.proj(concat_input)
        trans_out_logits = self.transformer(proj_out)
        return trans_out_logits

class MTP_and_deepseek(nn.Module, config):
    def __init__(self, config:Deepseekv3Config):
        super().__init__()
        self.share_embedding = share_embedding(config)
        self.share_lm_head = share_output_head(config)
        self.loss = nn.CrossEntropyLoss(ignore = config.pad_token_id)
        self.Model_trans_blocks = nn.ModuleList(
            [
                transformer_block(config, layer_idx)
                for layer_idx in range(config.num_hidden_layers)
            ]
        )
        self.MTP_trans_blocks = nn.ModuleList(
            [
                transformer_block(config, layer_idx)
                for layer_idx in range(config.num_MTP_layers)
            ]
        )
        self.config = config
        self.alpha_list = config.alpha_list#对于每一个MTPloss的加权列表
    def forward(self, input_ids):
        ## input_ids :
        ##[bos, tok1, tok2,..... last_tok]
        ##labels_origin:
        ##[tok1, tok2,....., last_tok, eos/pad]
        ##labels_MTP1
        ##[tok2,..... last_tok, eos, pad]
        ##labels_MTP2
        ##[tok3,..... last_tok, eos, pad, pad]
        embedding_logits = self.share_embedding(input_ids)
        deepseek_hidden = embedding_logits

```

```

for index,trans_block in enumerate(self.Model_trans_blocks):
    deepseek_hidden = trans_block(deepseek_hidden)
deepseek_logits = self.share_lm_head(deepseek_hidden)
labels = torch.cat([input_ids[:, 1:], torch.full((input_ids.size(0), 1), config.pad_token_id)], dim=1)
Main_loss = self.loss(deepseek_logits, labels)

last_mtp_out = deepseek_hidden
for ind, MTP in enumerate(self.MTP_trans_blocks):
    input_ids_trunc = torch.cat([input_ids[:, ind + 1 :], # 截取从 ind+1 开始的部分
                                torch.full((input_ids.size(0), ind + 1), self.config.pad_token_id), # 在后面添加 pad_token_id],
                                dim=1,)
    mtp_out = MTP(last_mtp_out, input_ids_trunc, self.share_embedding)
    mtp_logits = self.share_lm_head(mtp_out)
    last_mtp_out = mtp_out
    labels_trunc = torch.cat([input_ids_trunc[:, 1:], torch.full((input_ids.size(0), 1), config.pad_token_id)], dim=1)
    mtp_loss = self.loss(mtp_logits, labels_trunc)
    alpha = self.alpha_list[ind]
    Main_loss += alpha*mtp_loss
return Main_loss

```

## 3.2 专家负载均衡

### 3.2.1 辅助函数

### 3.2.2 本文方案

1. 对于每一个专家的亲和度（值越高越容易激活）加上一个偏置 b，如下图所示：

$$g'_{i,t} = \begin{cases} s_{i,t}, & s_{i,t} + b_i \in \text{Topk}(\{s_{j,t} + b_j | 1 \leq j \leq N_r\}, K_r), \\ 0, & \text{otherwise.} \end{cases}$$

知乎 @vehicle engineer

亲和度计算

1. 训练过程中，记录每个专家的负载率，通过调整偏置 b 进行循环，最终保证专家负载率平衡。

#以下 a、b、c 为专家，第一个数字为原始亲和度 Sit，第二个数字为偏置 bi，第三个数字为该专家的输出值。  
topk 为 2，output 为 MoE 层最终输出：

```

a: 0.4 - 0.1 100
b: 0.35 + 0 50
c: 0.25 + 0.1 120
moe_output = 50 * 0.35 + 0.25 * 120

```

## 3.3. 训练成本

在同等模型总参数量级下，训练成本降低一个量级。主要来源于以下几点：

### 3.1 混合精度：

- FP8+ 训练与 BF16 相比，理论计算量与通信量降低一倍。

### 3.2 MoE 层：

以下对同参数量的MLP和MOE进行计算量和通信量的对比

### 3.2.1原始 MLP 的计算量

- MLP 结构：
  - 两个矩阵：
    - 第一个矩阵：  $[h, 2.5h]$  。
    - 第二个矩阵：  $[2.5h, h]$  。
  - 每个 token 向量的计算量为：

$$\text{计算量} = h \times 2.5h + 2.5h \times h = 5h^2$$

### 3.2.2 MoE 的计算量

- MoE 结构：
  - 假设有  $n$  各专家，每次选用  $k$  个专家。
  - 每个专家的两个矩阵：
    - 第一个矩阵：  $[h/n^{1/2}, 2.5 * h/n^{1/2}]$  。
    - 第二个矩阵：  $[2.5 * h/n^{1/2}, h/n^{1/2}]$  。
  - 每个 token 每个专家的计算量为：

$$\text{计算量} = h/n^{1/2} \times 2.5h/n^{1/2} + 2.5h/n^{1/2} \times h/n^{1/2} = 2.5h^2/n + 2.5h^2/n = 5h^2/n$$

- $k$  各专家的平均每 token 计算量为：

$$\text{计算量} = k \times 5h^2/n = 5kh^2/n$$

### 3.2.3 MoE 与 MLP 的计算量对比

- MLP 计算量：

$$MLP\text{计算量} = 5h^2$$

- MoE 计算量：

$$MoE\text{计算量} = 5kh^2/n$$

- 计算量比例：

$$\text{比例} = MoE\text{计算量}/MLP\text{计算量} = 5kh^2/5h^2/n = k/n$$

- 由于  $k \ll n$ ，MoE 的计算量大大下降。

### 3.2.4 分布式训练的通信量

- MLP 通信量：
  - MLP 需要传输所有参数，通信量为 1（假设为单位通信量）。
- MoE 通信量：
  - MoE 每次仅选择  $k$  个专家，因此通信量下降到：
  - 通信量= $k/n$

### 3.2.5 总结

- MLP：
  - 计算量：  $5h^2$
  - 通信量： 1。



- **MoE:**
  - 计算量:  $5kh^2/n$
  - 通信量:  $k/n$ 。
- **优势:**
  - MoE 通过选择部分专家 (  $k$  个) 进行计算, 显著降低了计算量和通信量。
  - 由于  $k \ll n$ , MoE 的计算量和通信量都大大下降。

### 3.3.Dual 训练框架:

- 精心设计的框架可以在通信与计算并行, 减少了等待时间, 并且降低了气泡占比。

### 3.4 YARN+ 长文本拓展两阶段课程学习 (非原创) :

- YARN 是目前最常见的长文本拓展技术, 仅需 0.2% 的预训练量可以很好拓展文本长度。
- YARN 是 NTK-aware 的一种进阶形式。
- 具体是第一阶段将文本长度从 4k 拓展到 32k, 第二阶段从 32k 拓展到 128k。

### 3.5. 强化学习

#### Reward Model

1. 如果是代码、数学、agent等等, 使用规则和结果验证的方法获取Reward
2. 如果是自由形式但是有明确答案的任务, Reward Model 提供与正式答案是否相同的反馈
3. 如果是例如写作等等没有明确结果的任务, 我们使用Reward model进行输入和结果的打分
4. 奖励值应该分步给予, 而不是只对Final answer
5. 从deep seek v3的检查点初始化训练

#### RL

1. 使用GRPO算法进行在线-离线强化学习, 丢弃了critic模型, 减少显存并且加速
- 关于LLM的Critic一直困扰笔者, 原因是Critic是用来估计Value的, 也就是训练Critic的loss一直是用Critic的预测值和Value的mse

$$Critic_{loss} = Mse(V^{\pi}(\Theta) - Value)$$

- 然而Value是由reward累加获得的, 公式如下

$$V^{\pi}(s) = \mathbb{E}_{\pi} [\sum_{t=0}^{\infty} \gamma^t R_{t+1} \mid S_0 = s]$$

- R在LLM的PPO中又是由Reward model计算出来的

$$R_{(t+1)} = Rw(t + 1)$$

$$Critic_{loss} = Mse(V^{\pi}(\Theta) - Value) \\ = \mathbb{E}_{\pi} [Mse(V^{\pi}(\Theta) - \sum_{t=0}^{\infty} \gamma^t Rw(t + 1) \mid S_0 = s)]$$

- 因此Critic即在拟合Reward Model的期望值, 笔者理解使用Critic Model的唯一作用, Reward Model是用来模拟环境反馈, 一般难以得到很好的泛化, 所以使用Critic进行拟合能够减少方差。
- 2以下是GRPO使用组内相对优势进行优化的loss公式

与 DeepSeek-V2 (DeepSeek-AI, 2024c) 类似，我们采用组相对策略优化 (GRPO) (Shao et al., 2024)，它放弃了通常与策略模型大小相同的批评者模型，并估计取而代之的是来自小组分数的基线。具体来说，对于每个问题  $q$ ，GRPO 从旧策略模型  $\pi_{\theta_{old}}$  中采样一组输出  $\{o_1, o_2, \dots, o_G\}$ ，然后优化策略模型  $\pi_{\theta}$  通过最大化以下目标：

$$\begin{aligned} \mathcal{J}_{GRPO}(\theta) = & \mathbb{E}[q \sim P(Q), \{o_i\}_{i=1}^G \sim \pi_{\theta_{old}}(O|q)] \\ & \frac{1}{G} \sum_{i=1}^G \left( \min \left( \frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)} A_i, \text{clip} \left( \frac{\pi_{\theta}(o_i|q)}{\pi_{\theta_{old}}(o_i|q)}, 1 - \varepsilon, 1 + \varepsilon \right) A_i \right) - \beta \mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) \right), \end{aligned} \tag{26}$$
$$\mathbb{D}_{KL}(\pi_{\theta} || \pi_{ref}) = \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)} - \log \frac{\pi_{ref}(o_i|q)}{\pi_{\theta}(o_i|q)}$$

GRPO损失函数

- 3以上GRPO不再计算Critic估计的优势函数，而是优化每个组（每一个组代表ACTOR同输入下，不同的输出采样）内的reward
- 1. 以下每次采样获得的优势，是该次采样的奖励减去组内奖励均值并且处以方差，即标准化过程
- 2. 标准化过程的目的是：不同组内标准化有一个统一量纲：
- 举例如下第一组（代码任务）分数，5，4，3。第二组（作文任务）分数1，2，3。如果不做标准化，所有优势为正，更加重要的是，由于两组的整体均值为3，此时将导致第一组所有样本被奖励，第二组所有样本被惩罚。这种情况下完全不会使得模型变好。

其中  $\varepsilon$  和  $\beta$  是超参数；  $\pi_{ref}$  是参考模型；  $A_i$  是优势，源自与每个组内的输出相对应的奖励  $\{r_1, r_2, \dots, r_G\}$ ：

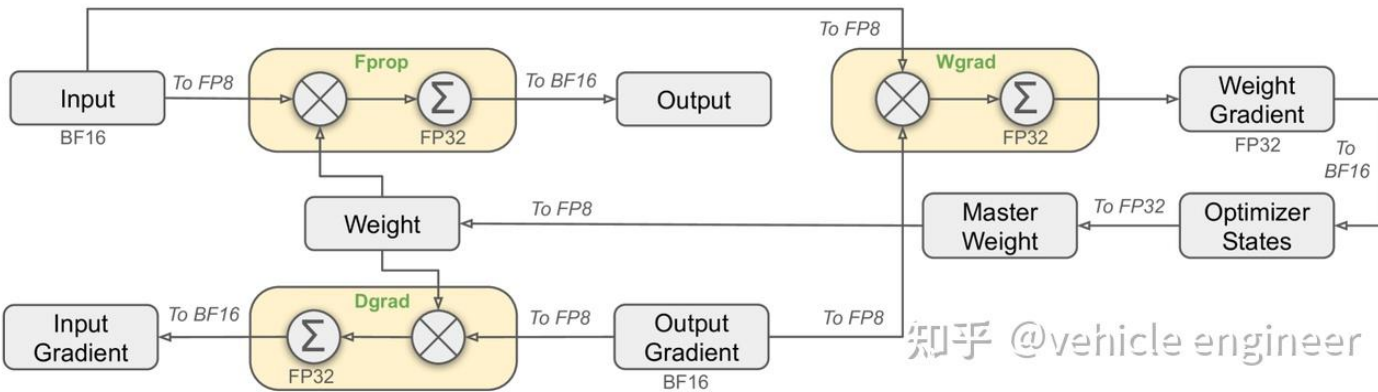
$$A_i = \frac{r_i - \text{mean}(\{r_1, r_2, \dots, r_G\})}{\text{std}(\{r_1, r_2, \dots, r_G\})}$$

GRPO的优势函数计算

- 4使用了多种提示词进行强化学习，减少了对SFT数据的依赖。（个人认为需要一个泛化能力优秀的reward model）

4. 分布式训练

4.1 FP8 混合精度训练



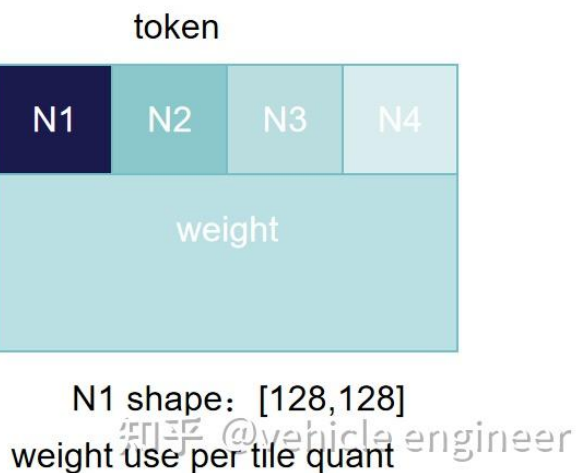
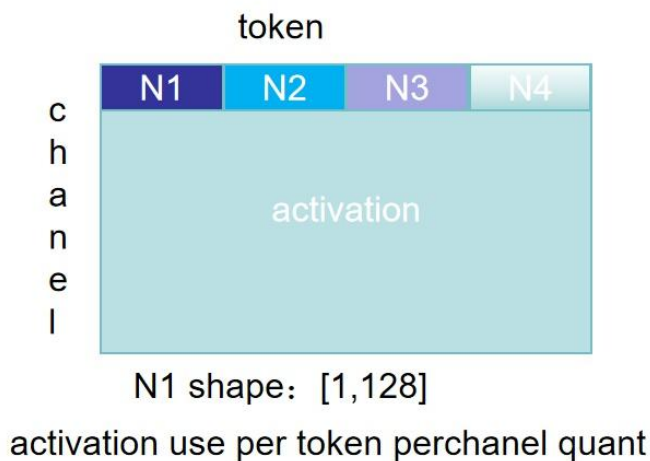
前、后向传播时的流程及其混合精度

FP8 的表示精度都要优于 INT8。

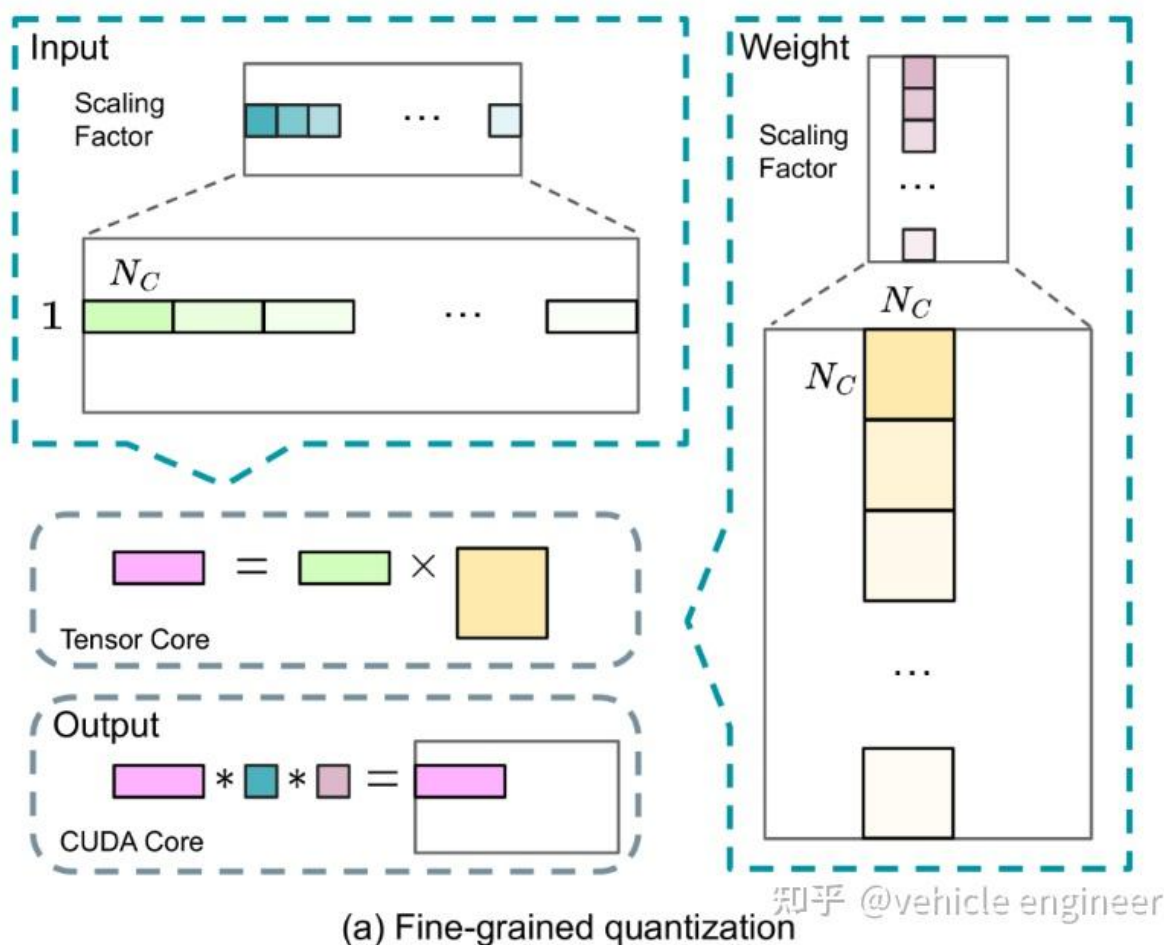
4.1.1 细粒度量化方法

本文中 BF16、FP32 向 FP8 转换采用分组量化，Activation 采用 per-token per-channel [1, 128]，Weight 采用 per-tile [128, 128]。

- 被量化向量：Weight: [7168, 4/3 \* 7168]
- Per-tensor: [7168, 4/3 \* 7168] -> 共享一个 SCALE, BIAS (最粗粒度)
- Per-token: [1, 7168 \* 4/3] -> 共享一个 SCALE, BIAS (粒度与 per-channel 类似，适合weight)
- Per-channel: [7168, 1] -> 共享一个 SCALE, BIAS (粒度与 per-token 类似，适合activation)
- Per-tile: [128, 128] -> 共享一个 SCALE, BIAS (粒度较细，精度较高)
- Per-token per-channel: -> 共享一个 [1, 128] (粒度最细，128 个值量化一次)



deepseek 量化分组示意图



原文量化图

- fp8的优势在于计算量与显存降低一倍

4.1.2 fp8精度下溢问题：

比如A（10000，4000）\*B（4000，10000），会得到一个C矩阵（10000，10000），每个值都要进行4000次乘法与加法，这种情况很容易造成精度下溢，因为要对齐指数位，所以尾数位要后移。举例如下：

```

    假设有两个 FP8 数：
    数 A：1.1251.125（二进制表示为 0 1000 001）
    数 B：0.06250.0625（二进制表示为 0 0111 000）

    1. 解码
    数 A：
    符号位：0（正数）
    指数位：1000=81000=8，实际指数 EA=8-7=1
    尾数位：001001，实际尾数 MA=1.001
    数值：1.125
    数 B：
    符号位：0（正数）
    指数位：0111=70111=7，实际指数 EB=7-7=0
    尾数位：000000，实际尾数 MB=1.000
    数值：0.0625

    2. 对齐指数
    EA=1，EB=0
    将数 B 的尾数右移 1 位：
    MB=0.1000MB=0.1000
    EB=1EB=1

    3. 尾数相加
    MA=1.001MA=1.001
    MB=0.1000MB=0.1000
    相加结果：Msum=1.1010

    4. 规范化
    尾数 Msum=1.1010Msum=1.1010 已经是规范化形式，无需调整。

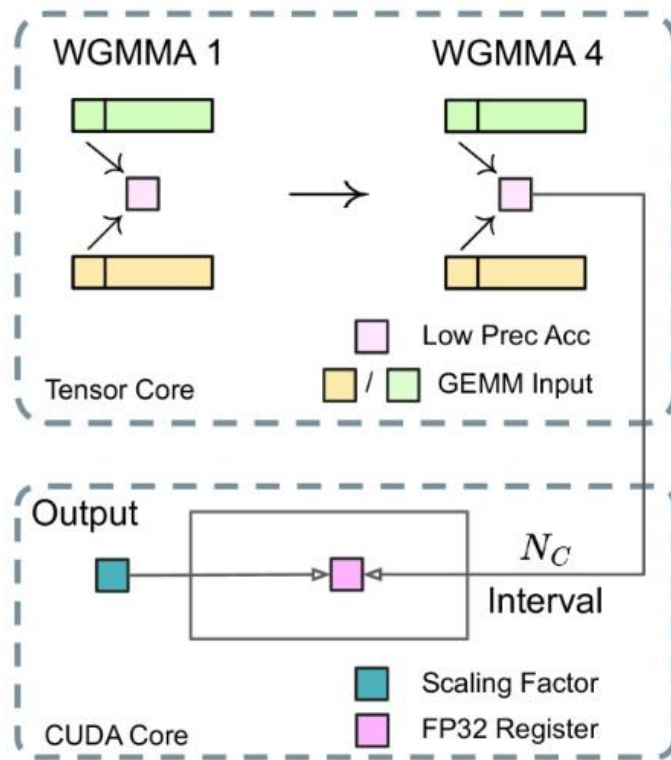
    5. 舍入
    FP8 的尾数只有 3 位，因此需要对 Msum=1.1010Msum=1.1010 进行舍入。
    使用向最近偶数舍入（Round to Nearest, Ties to Even）：
    舍入后的尾数：Msum=1.101

    6. 编码
    符号位：0（正数）
    指数位：1+7=8=10001+7=8=1000
    尾数为：101101
    最终 FP8 表示：0 1000 101
    数值：1.625
  
```

4.1.3 fp8大矩阵MMA算子精度下溢解决方案：

- 报告原文：

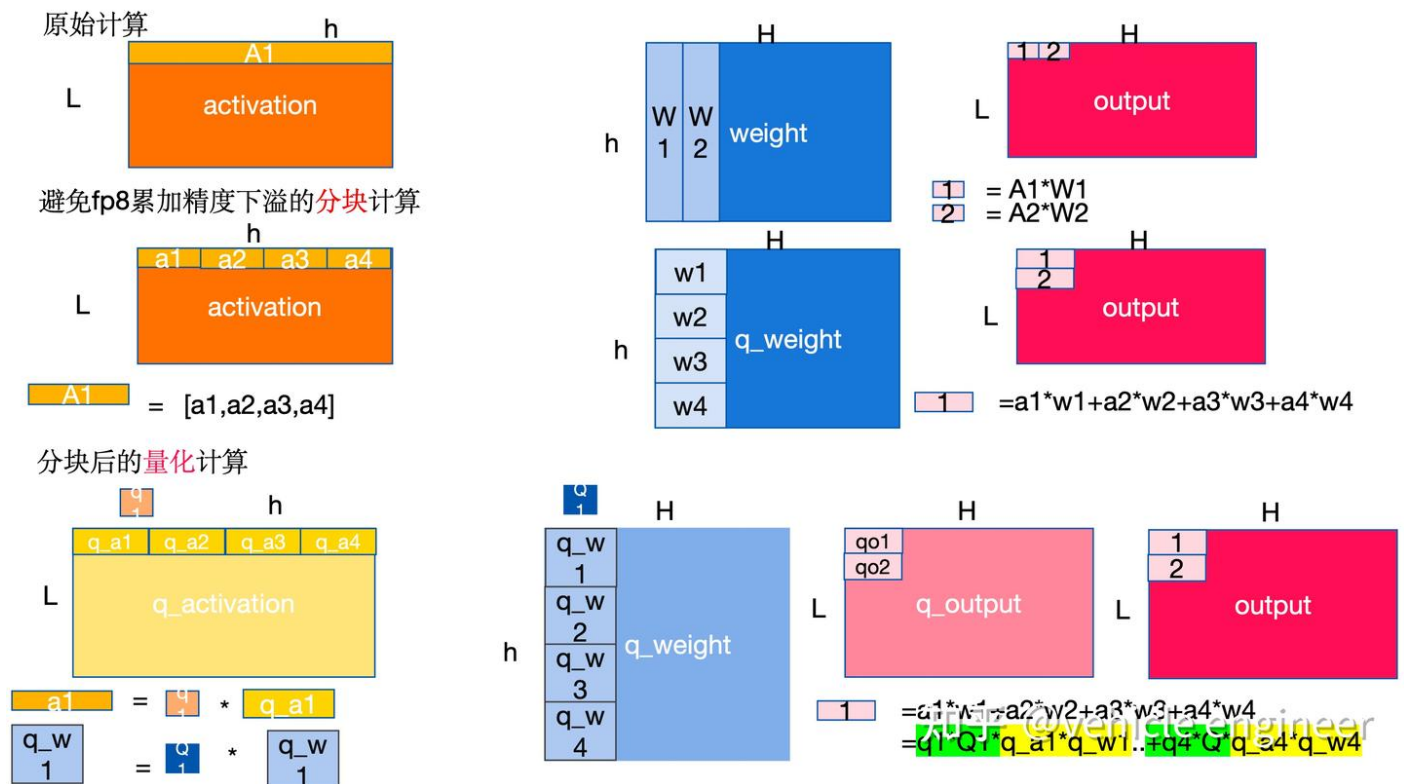
为了解决这一问题，我们采取了向CUDA Core升级以提高精度的策略（Thakkar et al., 2023年）。该过程如图7（B）所示。具体地说，在张量核上执行MMA（矩阵乘法累加）期间，使用有限的位宽累加中间结果。一旦达到间隔 NC，这些部分结果将被复制至CUDA内核上的FP32寄存器，在那里执行全精度FP32累加。如前所述，我们的细粒度量化沿内部维度K沿着应用每组缩放因子。这些缩放因子可以在CUDA内核上高效地相乘，作为具有最小附加计算成本的反量化过程。过程如下所示：



(b) Increasing accumulation precision

deepseek v3原图

- 个人理解:



fp8MMA防下溢算子及反量化过程

上图中右下角解读:

黄色代表Tensor core 算子: 速度快, 并行计算, 但是fp8累加存在精度下溢

绿色代表Cuda core 算子: 速度较慢, 但是fp32的精度高, 反量化和累加在这里做。

总的来说：量化后的分块的activation和weight在tensor core中进行MMA计算，得到的小块放到cuda core里面做累加以及反量化到fp32

#### 4.1.4 FP8 大矩阵 MMA防止精度下溢 操作步骤

##### 1. 做 MMA 操作时：

- 例如， `fp8_A[1000, 2560] * fp_W[2560, 200]` ， NC（量化数）为 128。

##### 2.做矩阵分解，遍例：

- 每个 A 中的 `[1, 128]` 和 W 中的 `[128, 128]` ，先使用 FP8 在 Tensor 核上算出结果 `C[1, 128]` 。
- 因为fp8大矩阵累加精度丢失，此时的分块后的矩阵仅仅累计 128 次，精度丢失较少。

##### 3.存储中间结果：

- 将 `C[1, 128]` 放到寄存器。
- 将 `fp8_A` 和 `fp8_W` 量化是对应的 `scalingA` 和 `scalingW` 也放到寄存器中。

##### 4.在 CUDA 核中进行反量化：

- 计算：

$$D_{fp32} = scalingA \times scalingB \times C_{fp8}$$

##### 5.循环重复步骤 2-4，直到矩阵分块完：

- 通过矩阵分解，重复前面的步骤 2-4。
- 最终将获得高精度的：

$$finalanswer_{[i,j]} = \sum D_{fp32}$$

```
## MMA防止精度下溢伪代码
def MMA_Mixed_Precision(A_fp8, W_fp8, scal_A, scal_W, NC=128):
    """
    使用混合精度（FP8 + FP32）进行矩阵乘法累加（MMA）操作。

    参数：
    - A_fp8: 形状为 [M, K] 的 FP8 激活值矩阵。
    - W_fp8: 形状为 [K, N] 的 FP8 权重矩阵。
    - scal_A: A_fp8 的量化缩放因子，形状为 [M, K // NC]。
    - scal_W: W_fp8 的量化缩放因子，形状为 [K // NC, N]。
    - NC: 量化分组大小，默认为 128。

    返回：
    - final_answer: 高精度的 FP32 结果矩阵，形状为 [M, N]。
    """
    import numpy as np

    # 获取输入矩阵的形状
    M, K = A_fp8.shape
    K, N = W_fp8.shape

    # 初始化最终结果矩阵
    final_answer = np.zeros((M, N), dtype=np.float32)

    # 遍历 A 的行
    for i in range(M): # 遍历 A 的每一行
        # 遍历 W 的列，以 NC 为步长
        for j in range(0, N, NC): # 遍历 W 的每一列，每次处理 NC 列
            # 初始化累加器，形状为 [1, NC]
```



```

D_fp32 = np.zeros((1, NC), dtype=np.float32)

# 遍历内部维度 K，以 NC 为步长
for k in range(0, K, NC):
    # 提取 A 的当前块 [1, NC]
    A_block = A_fp8[i, k:k+NC] # 形状为 [1, NC]

    # 提取 W 的当前块 [NC, NC]
    W_block = W_fp8[k:k+NC, j:j+NC] # 形状为 [NC, NC]

    # 在 Tensor 核上使用 FP8 计算矩阵乘法
    C_fp8 = np.dot(A_block, W_block) # 形状为 [1, NC]

    # 提取对应的缩放因子
    scale_A = scal_A[i, k // NC] # A 的缩放因子
    scale_W = scal_W[k // NC, j:j+NC] # W 的缩放因子，形状为 [1, NC]

    # 在 CUDA 核中进行反量化
    D_fp32 += scale_A * scale_W * C_fp8 # 累加到 D_fp32

# 将累加结果存储到最终答案矩阵
final_answer[i, j:j+NC] = D_fp32

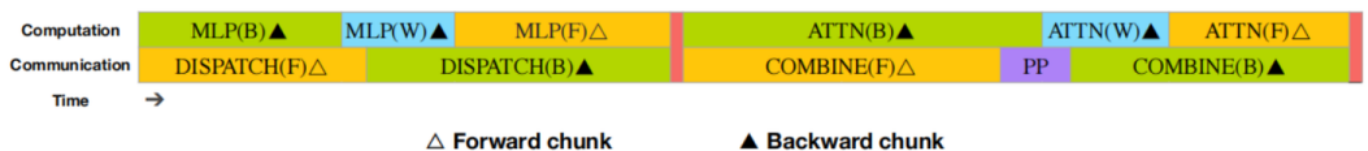
# 返回最终结果
return final_answer

```

## 4.2 DualPipe 框架

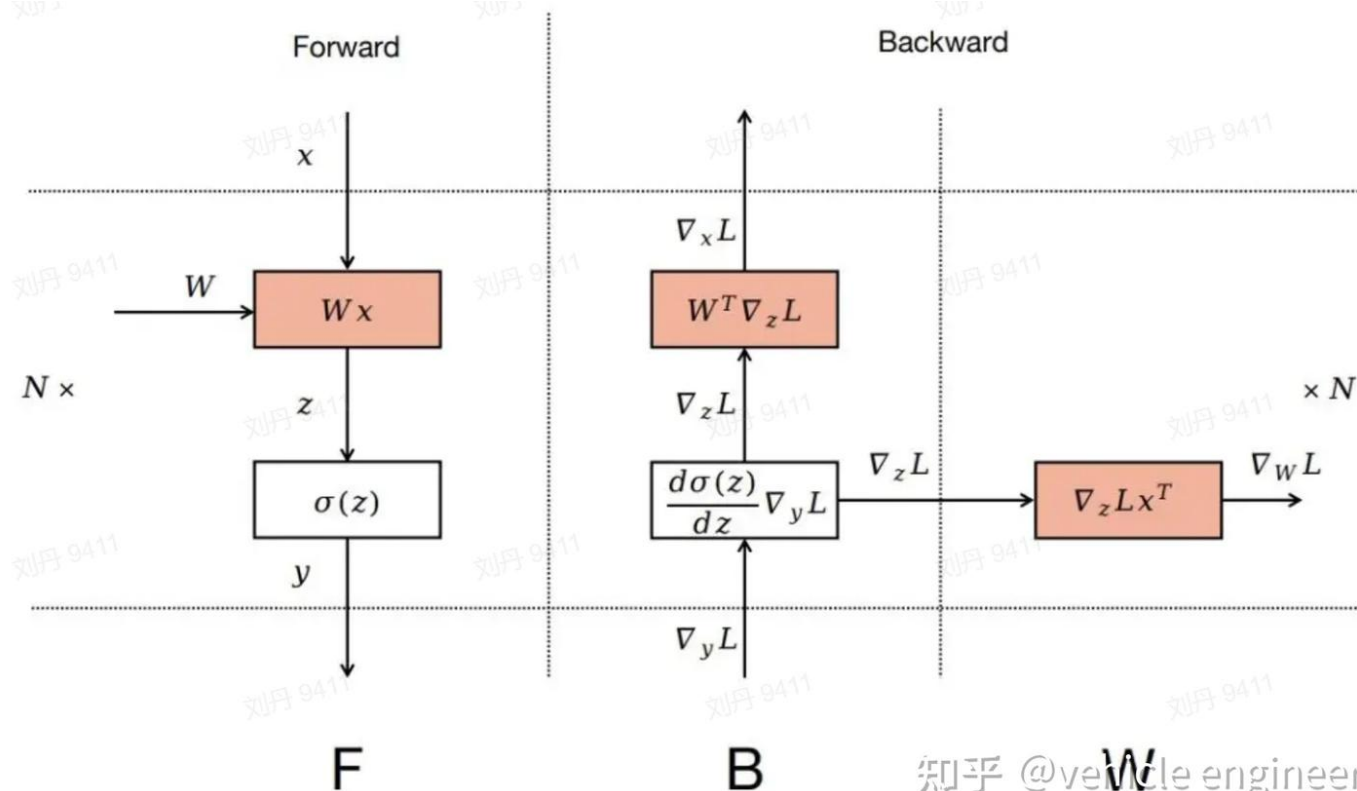
本部分参考了[deepseek预训练](#)

分布式训练的目的其实就一个：节省更多的资源，资源包括计算时间、显存、机器数量。总的来说应该是节省总的GPUhours。



通讯与计算同步示意图

上下图中的F（前向计算）、B（梯度计算）、W（跟新权重）含意一致



模型前向后向图

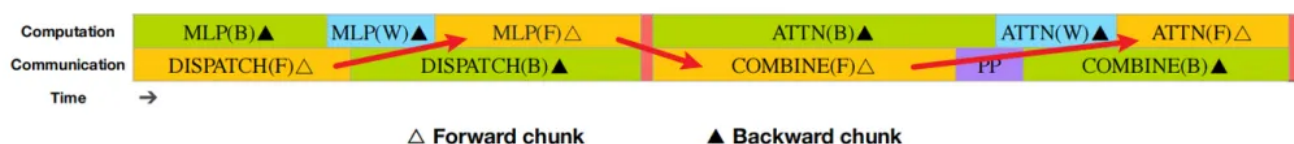
在模型训练中，主要计算量来源于 **ATTENTION- $O(L^2)$**  和 **MLP- $O(H^2)$** 。

由于分布式训练，前向和后向计算均需要通信，通信包括 **dispatch**（将输入分到各个 weight、expert）和 **combine**（将各个 weight、expert 的输出结果聚合）两部分。

在同一个 batch 中，通信和计算是交替进行的，这会导致效率低下。为此，DeepSeek V3 提出了双管路的方法，使得通信与计算能够并行

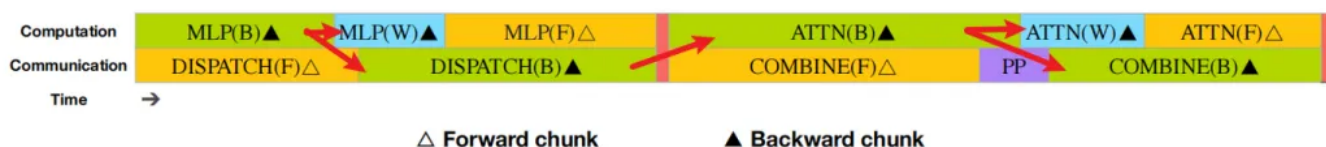
1. 模型训练中主要计算量来源于ATTENTION- $O(L^2)$ ，MLP- $O(H^2)$ 。
2. 由于分布式训练，导致前向后向计算均需要通信，通信包括dispatch（将输入分到各个weight、expert）、combine（将各个weight、expert的输出结果聚合）两部分。
3. 由于在同一个batch中，通信和计算是交替进行的，这将导致效率低下。为此deepseek v3 提出双管路的方法，使得通信与计算能够并行。

前向图如下：



前向传播

反向传播如下：



反向传播

## 1. 总体来说：

### 1. 双路的流水线并行

- DeepSeek V3 的双向 PP 调度中，还是 8 PP 为例：
- device 0 上有 Layer 0 以及 Layer 7 的权重
- device 1 上有 Layer 1 以及 Layer 6 的权重
- device 7 上有 Layer 7 以及 Layer 0 的权重
- 相当于有 2 份相同的模型副本切分在不同的层，Forward 的顺序可以从 device 0 到 7，也可以从 device 7 到 0。
- 这么设计的目的在于，下图中间部分存在极大范围的overlap backward&forward，由上面向前图和后向图可知，这样做可以减少时间。本质就是在每个device都有同时进行的第i层的前向和n-i层的反向传播，比如图中蓝色框中是4号数据在device4上的layer4进行前向传播，同时10号数据在device4的layer3上进行反向传播。



dual pipe解读

## 4.3 节省显存

### 4.3.1 Gradient Checkpoint

通过梯度检查点技术，减少显存占用。

### 4.3.2 将模型参数的指数移动平均值保存到 CPU

将模型参数的指数移动平均值保存到 CPU，减少 GPU 显存占用。

## 4.4 MTP 的 Share 模块

将 embedding、lmhead 等共享模块放在一起 PP 的 GPU 上，减少通信和显存占用。

## 5. 数据准备

### 5.1 预训练数据准备

准备预训练数据。

### 5.2 DeepSeek R1 蒸馏

1. 目标是平衡对于复杂问题的推理能力与对于一般问题过度思考，Deepseek R1是类似于gpt4 o1这种方法训练的复杂推理模型，运用了test time scaling技术
2. 将Deepseek R1的数据蒸馏出来，然后根据这些数据，考虑格式、长度等进行sft和RL获得data model
3. Data model 的sft数据构造为以下两类：
  1. 普通模式：<问题, 原始响应>
  2. R1模式<系统提示词, 问题, R1 响应>
4. 使用在线强化学习高温度对Data model进行采样，经过以上两种格式的sft后，就算高温度会生成融合两种形式的响应
5. 对第四步生成的数据进行强化学习，便能在普通温度下融合R1模式和普通模式，获得完整的Data model
6. 对于Data model做了一个拒绝采样的方式进行生成最终模型的sft数据，作为语言模型，非推理模型平衡了输出长度和复杂度。

## 6. 适配推理引擎

## 6.1 Prefilling (Compute Bound)

### 并行策略

- **Attention 模块**: TP4, 8DP, SP (4000 的训练长度需要 SP? )
- **MoE 模块**:
  - 32路ep并行, 专家总数256/专家并行数32=8, 即每个gpu有8+1 (冗余专家) 个专家:
  - 这样配置的原因是专家数一定时, 专家并行数小, 每张卡上专家数就大, 那每张卡上获得的token计算就多, 由于同一个卡内的专家计算过程中相互独立, 此时可以做并行计算, 从而达到充分利用算力的目的。
  - 另外需要注意的是Moe是特殊的MLP, 在prefilling阶段是可以并行的, 输入向量[B,L,H],可以看成一次型输入B\*L个token, 每个token占了[H]的向量, 然后向Moe层的专家去分配这些token。
- **节点间通信**: infinite band
- **节点内通信**: NVLink
- **浅层的 MLP**: 仅使用 1TP 并行, 因为 TP 的通信量最大, TP1 为最少通信量
- **两个微批次同时进行**: 一个微批次进行计算, 一个微批次进行通信, 从而覆盖通信时间

### 专家平衡策略

- 每张 GPU 部署 8 个专家, 并加上一个额外的高负载 (冗余) 专家
- 统计每个专家的负载, 每个 10 分钟重新设置冗余专家

## 6.2 Decoding (I/O Bound)

### 并行策略

- 40 各节点、320 个 GPU 组成
- **Attention 模块**: TP4、SP、DP80
- **MoE 部分**: EP320, 即每个 GPU 有一个专家
- **两个微批次同时进行**: 一个微批次进行 attention 计算, 一个微批次进行通信, 从而覆盖通信时间

编辑于 2025-01-31 17:19 · IP 属地湖南

DeepSeek-V3

来源: 知乎 | 悠趣谷 · 零成本 | 我要插件